Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра радіофізики та комп’ютерних технологій

Звіт

З лабораторної №5

З дисципліни «Професійний Python»

На тему **«Обробка помилок, файли та серіалізація»**

Виконав:
Студент 3 курсу
групи ФеП-31
Войтович М.А.
Перевірив:
проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Імпорт та експорт системи банківських рахунків)

Мета: Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних. Опанувати механізми винятків (try, except, raise, custom exceptions, exception chaining), принципи EAFP і fail-fast, контекст-менеджери (with, contextlib), модуль logging, сучасну роботу зі шляхами через pathlib, потокове читання (streaming I/O), роботу з CSV, JSON та YAML, а також забезпечення надійності за допомогою атомарного запису (atomic write).

Теоретичні відомості

Обробка винятків та кастомні класи помилок (Exception Handling):

Управління помилками в Python базується на конструкціях try, except, else, finally та ключовому слові raise. Для побудови архітектурно чистих додатків створюють ієрархії власних класів винятків (відповідно до базового класу Exception), що дозволяє чітко розрізняти бізнес-помилки, збої конфігурації та проблеми введення-виведення (I/O).

Зв’язування винятків (Exception Chaining):

Механізм перехоплення одного винятку та генерації іншого з явним збереженням контексту оригінальної помилки за допомогою синтаксису raise NewException(...) from error. Це забезпечує повний traceback і значно полегшує дебагування програм.

Принципи EAFP та Fail-Fast:

EAFP (Easier to Ask for Forgiveness than Permission) — стиль програмування в Python, за якого виконання операції відбувається без попередніх перевірок, а можливі помилки перехоплюються через блок try-except.

Fail-Fast — принцип проєктування, який вимагає негайного припинення роботи програми (або генерації критичної помилки) на найранішому етапі у разі виявлення порушення цілісності даних чи відсутності необхідних ресурсів (наприклад, конфігураційних файлів).

Робота з файловою системою через pathlib:

Сучасний модуль стандартної бібліотеки Python для об'єктно-орієнтованої роботи зі шляхами до файлів і директорій. На відміну від застарілого модуля os.path, pathlib.Path забезпечує кросплатформність, зручні методи відкриття файлів, перевірки існування та створення директорій.

Потокове введення-виведення (Streaming I/O та Generators):

Обробка великих файлів (наприклад, CSV) рядок за рядком або порціями за допомогою генераторів (yield, yield from) та ітераторів. Це дозволяє уникнути завантаження всього масиву даних в оперативну пам'ять (RAM), забезпечуючи високу ефективність і масштарування (Memory Efficiency).

Контекст-менеджери та Resource Management:

Конструкція with та протокол контекст-менеджерів (__enter__, __exit__ або декоратор @contextmanager із модуля contextlib) гарантують автоматичне звільнення системних ресурсів (закриття файлів, з'єднань із базами даних) та коректне вимірювання часу виконання блоків коду навіть у разі виникнення непередбачуваних винятків.
Структуроване логування (Logging):
Використання вбудованого модуля logging для фіксації подій різного рівня важливості (DEBUG, INFO, WARNING, ERROR, CRITICAL). Забезпечує виведення інформації у файли та консоль із чітким форматуванням часових міток, назв модулів та статусів виконання.
Атомарний запис (Atomic Write):
Патерн безпечного збереження файлів, який гарантує, що цільовий файл ніколи не буде пошкоджено (не запишеться наполовину) внаслідок збою живлення, аварійного завершення програми чи переповнення диску. Дані спочатку записуються у тимчасовий файл (наприклад, із розширенням .tmp), після чого за допомогою атомарної операції перейменування/заміни (os.replace) миттєво підміняють оригінальний файл.

Постановка задачі
Створити надійний модуль імпорту, експорту та конфігурації даних для системи банківських рахунків (Варіант 4).
Поля вхідних даних (CSV): account_number, client, balance, operation, amount.
Правила валідації:

1. Перевірка наявності та коректності номера рахунку та імені клієнта.
2. Баланс (balance >= 0).
3. Дозволені операції (deposit або withdraw).
4. Сума операції (amount > 0).
5. Захист від появи від'ємного балансу після операції списання (withdraw).

Технічні вимоги: використання pathlib, потокового читання CSV через генератори, ієрархії власних винятків із exception chaining, завантаження та перевірка YAML-конфігурації (PyYAML), логування подій із захистом конфіденційних даних, підтримка режимів strict / tolerant, а також атомарний експорт результатів у форматі JSON із використанням тимчасових файлів (.tmp).

Хід виконання

1. Структура проєкту (src-layout)

Проєкт організовано як модульний Python-пакет із чітким розділенням відповідальності компонентів:



2. Реалізація ключових компонентів

Ієрархія власних винятків (exceptions.py): Створено базовий клас ApplicationError, від якого успадковуються ConfigurationError та DataError. Для обробки помилок даних реалізовано DataImportError, DataExportError та RecordValidationError (з підтримкою атрибутів line_number і field). Усі помилки перетворюються з використанням exception chaining (from error), що зберігає початковий traceback.

```python
class ApplicationError(Exception):
    """Base application exception."""

class ConfigurationError(ApplicationError):
    """Configuration is invalid or missing."""

class DataError(ApplicationError):
    """Base data exception."""

class DataImportError(DataError):
    """Input data cannot be imported."""

class RecordValidationError(DataError):
    """Record fails validation rules."""
    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
    ) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field

class DataExportError(DataError):
    """Output cannot be written."""
```

Моделі даних та конфігурації (models.py): Використано заморожені датакласи (@dataclass(frozen=True, slots=True)) для представлення банківського рахунку (BankAccount) та структур конфігурації (AppConfig, ProcessingConfig, LoggingConfig).

```python
from dataclasses import asdict, dataclass
from pathlib import Path

@dataclass(frozen=True, slots=True)
class BankAccount:
    account_number: str
    client: str
    balance: float
    operation: str
    amount: float

    def to_dict(self) -> dict[str, object]:
        return asdict(self)

@dataclass(frozen=True, slots=True)
class ProcessingConfig:
    minimum_balance: float
    skip_invalid: bool

@dataclass(frozen=True, slots=True)
class LoggingConfig:
    level: str
    path: Path

@dataclass(frozen=True, slots=True)
class AppConfig:
    input_path: Path
    output_path: Path
    processing: ProcessingConfig
    logging: LoggingConfig
```

Конфігурація та валідація (config.py): Модуль завантажує параметри з файлу config.yaml за допомогою yaml.safe_load(). Реалізовано перевірку наявності вхідного файлу та валідність параметрів (принцип *fail-fast*).

```python
from pathlib import Path
import yaml

from bank_system.exceptions import ConfigurationError
from bank_system.models import AppConfig, LoggingConfig, ProcessingConfig

def load_config(path: Path) -> AppConfig:
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Configuration file not found: {path}") from error
    except yaml.YAMLError as error:
        raise ConfigurationError("Invalid YAML configuration") from error

    try:
        config = AppConfig(
            input_path=Path(raw["input"]["path"]),
            output_path=Path(raw["output"]["path"]),
            processing=ProcessingConfig(
                minimum_balance=float(raw["processing"]["minimum_balance"]),
                skip_invalid=bool(raw["processing"]["skip_invalid"]),
            ),
            logging=LoggingConfig(
                level=str(raw["logging"]["level"]),
                path=Path(raw["logging"]["path"]),
            ),
        )
    except (KeyError, TypeError, ValueError) as error:
        raise ConfigurationError("Configuration structure is invalid") from error

    validate_config(config)
    return config

def validate_config(config: AppConfig) -> None:
    if not config.input_path.exists():
        raise ConfigurationError(f"Input file not found: {config.input_path}")
```

Потоковий читач (readers.py): Використовує pathlib.Path та csv.DictReader у поєднанні з генераторами (yield from / yield), що дозволяє обробляти великі файли без завантаження всього масиву в оперативну пам'ять (streaming I/O).

```python
import csv
from collections.abc import Iterator
from pathlib import Path

from bank_system.exceptions import DataImportError

def read_csv_rows(path: Path) -> Iterator[tuple[int, dict[str, str]]]:
    try:
        with path.open("r", encoding="utf-8", newline="") as file:
            reader = csv.DictReader(file)
            for line_number, row in enumerate(reader, start=2):
                yield line_number, row
    except OSError as error:
        raise DataImportError(f"Cannot read input file: {path}") from error
```

Валідація бізнес-правил Варіанта 4 (validators.py): Функція parse_bank_account перевіряє правильність полів рахунку, тип операції (deposit / withdraw), додатність суми та гарантує відсутність ситуації, коли баланс після списання стає від'ємним. У разі помилки генерується RecordValidationError.

```python
from bank_system.exceptions import RecordValidationError
from bank_system.models import BankAccount

def parse_bank_account(row: dict[str, str], *, line_number: int) -> BankAccount:
    try:
        account_number = str(row["account_number"]).strip()
        if not account_number:
            raise ValueError("Account number is empty")
    except (KeyError, ValueError) as error:
        raise RecordValidationError("Invalid account number", line_number=line_number, field="account_number") from error

    client = str(row.get("client", "")).strip()
    if not client:
        raise RecordValidationError("Client name is empty", line_number=line_number, field="client")

    try:
        balance = float(row["balance"])
    except (KeyError, TypeError, ValueError) as error:
        raise RecordValidationError("Invalid balance value", line_number=line_number, field="balance") from error

    if balance < 0:
        raise RecordValidationError("Balance cannot be negative", line_number=line_number, field="balance")

    operation = str(row.get("operation", "")).strip().lower()
    if operation not in ("deposit", "withdraw"):
        raise RecordValidationError("Invalid operation (must be deposit or withdraw)", line_number=line_number, field="operation")

    try:
        amount = float(row["amount"])
    except (KeyError, TypeError, ValueError) as error:
        raise RecordValidationError("Invalid amount value", line_number=line_number, field="amount") from error

    if amount <= 0:
        raise RecordValidationError("Amount must be greater than zero", line_number=line_number, field="amount")
```

```python
# Контроль недопущення від'ємного балансу після операції списання
final_balance = balance
if operation == "deposit":
    final_balance += amount
elif operation == "withdraw":
    if balance - amount < 0:
        raise RecordValidationError("Insufficient funds: operation results in negative balance", line_number=line_number, field="amount")
    final_balance -= amount

return BankAccount(
    account_number=account_number,
    client=client,
    balance=round(final_balance, 2),
    operation=operation,
    amount=amount,
)
```

Сервісний шар та статистика (services.py): Клас ImportStatistics відстежує загальну кількість записів, кількість валідних та невалідних рядків. Підтримується робота в режимі tolerant (пропуск невалідних рядків з попередженням у логах) та strict (зупинка на першій помилці).

```python
from collections.abc import Iterator
from dataclasses import dataclass
from pathlib import Path
import logging

from bank_system.exceptions import RecordValidationError
from bank_system.models import BankAccount
from bank_system.readers import read_csv_rows
from bank_system.validators import parse_bank_account

logger = logging.getLogger(__name__)

@dataclass
class ImportStatistics:
    total: int = 0
    valid: int = 0
    invalid: int = 0

def import_accounts(
    path: Path,
    *,
    skip_invalid: bool,
    statistics: ImportStatistics,
) -> Iterator[BankAccount]:
    for line_number, row in read_csv_rows(path):
        statistics.total += 1
        try:
            account = parse_bank_account(row, line_number=line_number)
        except RecordValidationError as error:
            statistics.invalid += 1
            logger.warning("Invalid record at line %s: %s", error.line_number, error)
            if skip_invalid:
                continue
            raise
        statistics.valid += 1
        yield account
```

Атомарний експортер (exporters.py): Експорт даних у JSON реалізовано через потоковий запис у тимчасовий файл із розширенням .tmp. Після успішного завершення запису файл атомарно замінює цільовий файл за допомогою методу .replace(), що унеможливлює пошкодження результатів у разі аварійного завершення програми.

```python
from collections.abc import Iterable
import json
from pathlib import Path

from bank_system.exceptions import DataExportError
from bank_system.models import BankAccount

def export_accounts_json(accounts: Iterable[BankAccount], path: Path) -> int:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    count = 0
    try:
        with temporary.open("w", encoding="utf-8") as file:
            file.write("[\n")
            first = True
            for account in accounts:
                if not first:
                    file.write(",\n")
                json.dump(account.to_dict(), file, ensure_ascii=False, indent=2)
                first = False
                count += 1
            file.write("\n]\n")
        temporary.replace(path)
    except (OSError, TypeError) as error:
        temporary.unlink(missing_ok=True)
        raise DataExportError(f"Cannot export data to {path}") from error
    return count
```

Контекст-менеджер та логування (utils.py, logging_config.py): Створено власний контекст-менеджер logged_operation на базі @contextmanager для заміру часу виконання операцій та їх логування. Модуль logging налаштовано з виведенням у файл та консоль, забезпечуючи заборону на запис конфіденційних реквізитів.

utils.py

```python
from contextlib import contextmanager
import logging
from time import perf_counter

logger = logging.getLogger(__name__)

@contextmanager
def logged_operation(operation: str):
    logger.info("%s started", operation)
    start = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("%s failed", operation)
        raise
    else:
        elapsed = perf_counter() - start
        logger.info("%s finished in %.6f s", operation, elapsed)
```

logging_config.py

```python
import logging
from pathlib import Path

def configure_logging(level: str, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    numeric_level = getattr(logging, level.upper(), logging.INFO)
    logging.basicConfig(
        level=numeric_level,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
        handlers=[
            logging.FileHandler(path, encoding="utf-8"),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

3. Робота з Git

Для відстеження змін у п’ятій лабораторній роботі було виконано серію логічних комітів:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add config.yaml src/bank_system/exceptions.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(core): додати конфігурацію YAML та ієрархію кастомних винятків"
[main 30fa099] feat(core): додати конфігурацію YAML та ієрархію кастомних винятків
 2 files changed, 37 insertions(+)
 create mode 100644 config.yaml
 create mode 100644 src/bank_system/exceptions.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/models.py src/bank_system/logging_config.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(models): реалізувати моделі даних та налаштування безпечного логування"
[main 78a13ae] feat(models): реалізувати моделі даних та налаштування безпечного логування
 2 files changed, 42 insertions(+), 96 deletions(-)
 create mode 100644 src/bank_system/logging_config.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/readers.py src/bank_system/validators.py src/bank_system/services.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(services): реалізувати потоковий читач CSV, валідацію рахунків та сервісний шар"
[main e1c0a3d] feat(services): реалізувати потоковий читач CSV, валідацію рахунків та сервісний шар
 3 files changed, 101 insertions(+), 104 deletions(-)
 create mode 100644 src/bank_system/readers.py
 create mode 100644 src/bank_system/validators.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/exporters.py src/bank_system/utils.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(io): реалізувати атомарний експорт у JSON та контекст-менеджер часу"
[main 9460195] feat(io): реалізувати атомарний експорт у JSON та контекст-менеджер часу
 2 files changed, 45 insertions(+)
 create mode 100644 src/bank_system/exporters.py
 create mode 100644 src/bank_system/utils.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/main.py data/accounts.csv README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "feat(main): підключити точку входу та завершити реалізацію пайплайну"
[main b2f363a] feat(main): підключити точку входу та завершити реалізацію пайплайну
 3 files changed, 63 insertions(+), 57 deletions(-)
 create mode 100644 data/accounts.csv
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git push -u origin main
remote: Repository not found.
fatal: repository 'https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab05.gi/' not found
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git remote set-url origin https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab05.git
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git push -u origin main
Enumerating objects: 157, done.
Counting objects: 100% (157/157), done.
Delta compression using up to 12 threads
Compressing objects: 100% (132/132), done.
Writing objects: 100% (157/157), 2.33 MiB | 1.41 MiB/s, done.
Total 157 (delta 55), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (55/55), done.
To https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab05.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Перевірка історії комітів (git log --oneline):

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git log --oneline
b2f363a (HEAD -> main, origin/main) feat(main): підключити точку входу та завершити реалізацію пайплайну
9460195 feat(io): реалізувати атомарний експорт у JSON та контекст-менеджер часу
e1c0a3d feat(services): реалізувати потоковий читач CSV, валідацію рахунків та сервісний шар
78a13ae feat(models): реалізувати моделі даних та налаштування безпечного логування
30fa099 feat(core): додати конфігурацію YAML та ієрархію кастомних винятків
1ec774f docs: додано звіт до лабораторної роботи №4
3c82a80 docs: оновлено README.md відповідно до архітектури Лабораторної роботи №4
8820214 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
c0891d7 refactor(main): оновлення точки входу програми відповідно до вимог Варіанта 4
1916232 feat(services): реалізація BankService з використанням Dependency Injection та ConsoleNotifier
9d4c57d feat(dto): додавання TypedDict для безпечної типізації вхідних даних
b9d02f7 feat(repositories): реалізація дженерика InMemoryRepository[T] з обмеженим TypeVar
9014b7a feat(protocols): створення runtime_checkable протоколу Notifier для структурної типізації
b6b22dd feat(models): реалізація Customer, ієрархії Account (Savings/Current) та кастомної колекції AccountCollection
cdad793 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
22f9ff4 Merge branch 'main' of https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab03
338f789 docs: оновлено README.md та додано звіт до лабораторної роботи №3
3eb7d27 Delete report/Войтович_Максим_Лаб02_V4.pdf
94e177e feat(main): інтегровано аналітику itertools, пакетну обробку та порівняння eager/lazy підходів
f01d91b feat(streaming): додано генератор тестового датасету, потоковий reader та lazy pipeline
8f6ded5 feat(iterators): реалізовано власний ітератор AccountRangeIterator та рекурсивний генератор з yield from
c7b759c Збереження поточного стану проєкту
5c2f55e Збереження поточного стану проєкту
f792e90 Звіт для 2 лабораторної
651213d Add README with project details and structure
0e5a2aa Оновлено main.py з інтерактивним меню для ЛР2
a155460 Додано обробку індексів, замикання, сортування та аналітику з defaultdict і *args
64eab4b Додано модуль data.py з **kwargs та модуль декораторів decorators.py
d7bef70 Додано звіт до лабораторної роботи №1
53d3b57 Додано точку входу та документацію застосунку Bank System
603e965 Додано модель банківського рахунку та бізнес-логіку
454af57 Створення структури проєкту
```

Індивідуальне завдання (Варіант 4). Надійний пайплайн обробки, валідації та експорту даних банківської системи

Розроблено надійну, безпечну та модульну систему обробки даних для управління банківськими рахунками відповідно до індивідуального варіанта №4. Проєкт реалізує багаторівневу архітектуру з використанням потокового

введення-виведення, гнучкої ієрархії винятків, конфігурації через YAML та безпечного атомарного експорту.

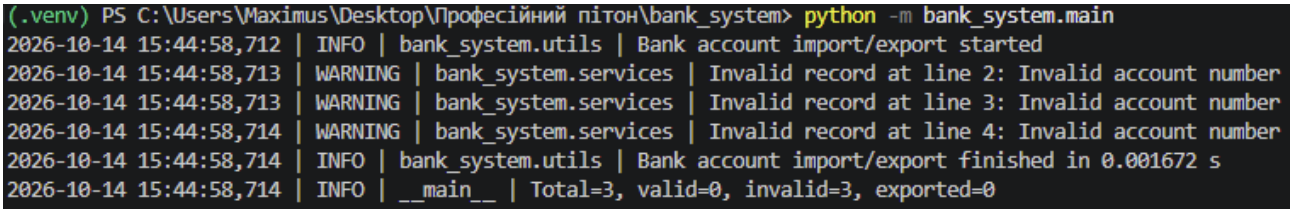
Архітектура системи та реалізовані компоненти:

1. **Ієрархія винятків та Exception Chaining (exceptions.py):** Створено розгалужену систему власних винятків (ApplicationError, DataError, DataImportError, RecordValidationError, ConfigurationError, DataExportError) із додатковими атрибутами (line_number, field) та збереженням початкового трасування за допомогою конструкції from error.
2. **Конфігурація та налаштування (config.yaml, config.py, logging_config.py):** Реалізовано завантаження налаштувань із зовнішнього файлу через безпечний yaml.safe_load(), їхню валідацію на старті (fail-fast), а також настроюване безпечне логування (logging) із записом у файл і виведенням у консоль.
3. **Моделі даних та типи (models.py):** Створено незмінні датакласи (BankAccount, AppConfig, ProcessingConfig, LoggingConfig) із підтримкою frozen=True та slots=True для оптимізації пам'яті й захисту стану.
4. **Потоковий імпорт CSV (readers.py, services.py):** Реалізовано потокове читання великих файлів за допомогою генераторів (Iterator та csv.DictReader), що унеможливлює перевантаження оперативної пам'яті.
5. **Валідація бізнес-правил (Варіант 4) (validators.py):** Налаштовано сувору перевірку вхідних даних (формат рахунків, непусті поля, додатні суми транзакцій, допустимі операції deposit/withdraw) та захист від появи від'ємного балансу після операції списання.
6. **Атомарний експорт даних (exporters.py):** Реалізовано безпечний механізм експорту у формат JSON через тимчасові файли (.tmp) із наступною атомарною замінкою (replace), що гарантує цілісність даних у разі збою програми.
7. **Контекст-менеджери та утиліти (utils.py):** Створено власний генераторний контекст-менеджер logged_operation за допомогою декоратора @contextmanager для точного вимірювання часу виконання та логування етапів роботи пайплайну.

Результат: Створено повноцінний, стійкий до помилок та оптимізований пайплайн обробки даних банківської системи, який повністю задовольняє всі вимоги методичних вказівок для лабораторної роботи №5.

Експериментальна частина

Було успішно прочитано тестовий датасет із файлу data/accounts.csv, виконано потокову валідацію транзакцій (з виявленням некоректного рядка з недостатньою кількістю коштів для списання), проведено атомарний експорт у output/accounts.json та зафіксовано всю статистику й події у файл логів logs/application.log.



Таблиця виконання вимог

Вимога лабораторної роботи / Індивідуального завдання	**Стан виконання**	**Деталі реалізації у проєкті**
Ієрархія власних винятків (Custom Exceptions)	**Виконано**	**Створено базовий клас ApplicationError та похідні ConfigurationError, DataImportError, DataExportError, RecordValidationError.**
Зв'язування винятків (Exception Chaining)	**Виконано**	**Усі системні помилки перехоплюються та прокидаються через конструкцію from error для збереження оригінального traceback.**
Принципи EAFP та Fail-Fast	**Виконано**	**Реалізовано перевірку наявності конфігурації та файлів на старті програми за принципом *fail-fast*.**
Робота з файловою системою через pathlib	**Виконано**	**Усі шляхи до файлів, папок (data, output, logs) та роботу з ними реалізовано за допомогою pathlib.Path.**
Потокове читання даних (Streaming I/O)	**Виконано**	**Використано генератори (yield, yield from) та csv.DictReader для читання великих файлів без завантаження в RAM.**
Конфігурація через YAML	**Виконано**	**Налаштування програми завантажуються з файлу**

Вимога лабораторної роботи / Індивідуального завдання	Стан виконання	Деталі реалізації у проєкті
		config.yaml за допомогою безпечного методу yaml.safe_load().
Валідація бізнес-правил Варіанта 4	Виконано	Перевірено поля рахунків, клієнтів, невід'ємність балансу, тип операції (deposit / withdraw), додатність суми та захист від мінусового балансу.
Режими обробки помилок (Strict / Tolerant)	Виконано	Підтримується режим толерантності (пропуск і логування невалідних рядків зі збором статистики) та суворий режим.
Атомарний експорт даних (Atomic Write)	Виконано	Результати спочатку записуються у тимчасовий файл .tmp, після чого атомарно замінюють цільовий JSON-файл через .replace().
Власний контекст-менеджер часу	Виконано	Створено контекст-менеджер на базі @contextmanager для заміру часу виконання пайплайну та логування.
Структуроване логування (Logging)	Виконано	Налаштовано модуль logging із фіксацією подій у консоль та файл із забороною логування конфіденційних даних.
Пайплайн та звітність (Statistics)	Виконано	Підраховується загальна кількість записів, кількість валідних, невалідних та успішно експортованих об'єктів.

1. Що таке exception?

Це об'єкт класу Exception (або його нащадків), який сигналізує про виникнення помилки або нештатної ситуації під час виконання програми.

2. Для чого використовується try?

Для визначення блоку коду, в якому потенційно можуть виникнути винятки, щоб перехопити їх та обробити.

3. Для чого except?

Для перехоплення певних типів винятків, що виникли в блоці try, та виконання альтернативного сценарію (обробки помилки).

4. Коли виконується else після try?

Тоді, коли в блоці try не було виявлено жодних винятків.

5. Коли виконується finally?

Завжди (незалежно від того, чи був виняток, чи успішно завершився блок try), зазвичай використовується для звільнення ресурсів.

6. Для чого використовується raise?

Для штучної генерації (виклику) винятку в коді.

7. Що означає raise без аргументів?

Повторне збудження (re-raising) поточного винятку всередині блоку except із збереженням його початкового контексту та traceback.

8. Чому не рекомендується except: pass?

Це «ковтає» (приховує) абсолютно всі помилки включно з системними (KeyboardInterrupt, SystemExit), ускладнюючи дебагування та приховуючи реальні проблеми в коді.

9. Чому потрібно перехоплювати конкретні exception types?

Щоб обробляти лише очікувані помилки (наприклад, FileNotFoundError), не маскуючи непередбачувані збої в інших частинах коду.

10. Що таке exception hierarchy?

Деревоподібна структура класів винятків у Python, де базовим є BaseException, а загальним для логічних помилок — Exception.

11. Для чого створювати custom exceptions?

Щоб чітко ідентифікувати специфічні бізнес-помилки або доменні збої, відрізняючи їх від стандартних помилок Python.

12.Що таке exception chaining?
Механізм зв'язування винятків, коли новий виняток генерується в контексті попереднього, зберігаючи ланцюжок причин.

13.Що означає raise ... from ...?
Синтаксис явного зв'язування винятків (raise NewError from original_error), який прикріплює оригінальний виняток як атрибут __cause__.

14.Яку перевагу дає chaining для debugging?
Дозволяє бачити повний traceback і розуміти першопричину збою (яка саме низькорівнева помилка викликала високорівневу).

15.Що означає EAFP?
"Easier to Ask for Forgiveness than Permission" (Легше попросити вибачення, ніж дозволу) — підхід, де дія виконується одразу, а помилки обробляються через try-except.

16.Що означає LBYL?
"Look Before You Leap" (Дивись, перш ніж стрибнути) — підхід із попередньою перевіркою умов (наприклад, if os.path.exists(...)).

17.Що таке fail-fast?
Принцип проєктування, за якого програма повинна негайно припинити роботу (або викликати помилку) при виявленні невідповідності або збою на старті.

18.Де доцільно застосовувати fail-fast?
Під час ініціалізації додатків, завантаження конфігурації, перевірки обов'язкових параметрів та з'єднань.

19.Що таке context manager?
Об'єкт, який керує життєвим циклом ресурсів за допомогою протоколу контексту (__enter__ і __exit__).

20.Для чого використовується with?
Для безпечного керування ресурсами, забезпечуючи їх обов'язкове відкриття та закриття/очищення.

21.Які methods утворюють context manager protocol?
Магічні методи __enter__() та __exit__().

22.Що робить enter()?
Виконує початкові дії перед входом у блок with та повертає керований ресурс.

23.Що робить exit()?
Виконує фінальні дії (очищення, закриття файлу) при виході з блоку with, навіть якщо сталася помилка.

24.Чи закриває with файл при exception?
Так, гарантовано закриває файл завдяки виклику методу __exit__.

25.Для чого contextlib.contextmanager?
Декоратор, який дозволяє перетворити звичайну генераторну функцію на повноцінний контекст-менеджер за допомогою yield.

26.Яку роль має yield у generator-based context manager?
Розділяє код на частину до __enter__ (перед yield) і частину після __exit__ (після yield).

27.Що таке pathlib.Path?
Класс із модуля pathlib для зручної об'єктно-орієнтованої роботи зі шляхами до файлів та директорій.

28.Чому pathlib кращий за ручну конкатенацію шляхів?
Забезпечує кросплатформність (автоматично адаптує слеші для Windows/Linux/macOS) та має інтуїтивні методи (/, .read_text(), .exists()).

29.Для чого задається encoding="utf-8"?
Щоб гарантувати коректне читання та запис символів кирилиці й інших мов без прив'язки до локального кодування системи.

30.Що таке CSV?
"Comma-Separated Values" — текстовий формат для зберігання та обміну табличними даними, де поля розділені комами (або іншими розділювачами).

31.Для чого csv.reader?
Для простого покрокового читання рядків CSV-файлу у вигляді списків рядків.

32.Для чого csv.DictReader?
Для читання CSV-файлу, де кожен рядок автоматично перетворюється на словник, де ключі — це заголовки колон.

33.Чим DictReader зручний для structured data?
Дозволяє звертатися до даних за назвами колонок (row["balance"]), а не за числовими індексами.

34.Для чого csv.DictWriter?

Для запису структурованих даних (словників) у формат CSV на основі визначених заголовків.

35.Що таке JSON?
"JavaScript Object Notation" — легкий текстовий формат обміну даними на основі синтаксису об'єктів та масивів JavaScript.

36.Що таке serialization?
Процес перетворення об'єкта в пам'яті (наприклад, словника чи датакласу) у послідовність байтів або текстовий формат (JSON, YAML тощо) для збереження чи передачі.

37.Що таке deserialization?
Процес відновлення об'єкта в пам'яті з текстового чи байтового представлення (наприклад, парсинг JSON-файлу).

38.Чим json.dump() відрізняється від json.dumps()?
json.dump() записує дані безпосередньо у файловий об'єкт, а json.dumps() повертає результат у вигляді рядка (str).

39.Чим json.load() відрізняється від json.loads()?
json.load() читає та десеріалізує дані з файлового об'єкта, а json.loads() — із текстового рядка.

40.Що таке JSONDecodeError?
Виняток, який виникає при спробі десеріалізувати зіпсований або невалідний JSON-рядок.

41.Для чого використовується YAML?
Для зручного читання людиною конфігураційних файлів та структур даних.

42.Чому рекомендується yaml.safe_load()?
Запобігає вразливостям безпеки (виконанню довільного доменного коду/об'єктів), які можливі при використанні небезпечного yaml.load().

43.Які типові сценарії використання JSON?
API-комунікації, вебсервіси, зберігання структурованих логів і невеликих баз даних у файлах.

44.Які типові сценарії використання YAML?
Файли конфігурацій додатків, CI/CD пайплайни (GitHub Actions, Docker Compose).

45.Що таке configuration file?
Зовнішній файл із налаштуваннями програми (параметри підключення, шляхи, режими роботи).

46.Чому configuration краще не hard-code у Python?
Змушує перекомпілювати/змінювати код при кожній зміні середовища (наприклад, перехід з розробки на production).

47.Що потрібно перевіряти у configuration?
Наявність файлу, повноту обов'язкових полів, типи даних та коректність значень (наприклад, діапазони чисел).

48.Що таке logging?
Систематизований процес фіксації подій, помилок та стану програми під час її виконання.

49.Чим logging відрізняється від print()?
Надає рівні важливості (levels), налаштування форматування, можливість запису у файли та гнучку фільтрацію без зміни коду.

50.Які існують logging levels?
DEBUG, INFO, WARNING, ERROR, CRITICAL.

51.Для чого logger.info()?
Для запису загальних інформаційних повідомлень про нормальний хід виконання програми.

52.Для чого logger.warning()?
Для фіксації попереджень про потенційні проблеми, які поки не зупиняють роботу програми.

53.Для чого logger.error()?
Для фіксації серйозних помилок, через які певна операція не вдалася.

54.Для чого logger.exception()?
Для автоматичного логування повідомлення рівня ERROR разом із повним traceback поточного винятку (викликається всередині except).

55.Що містить traceback?
Послідовність викликів функцій (стек викликів), яка привела до виникнення помилки, із зазначенням файлів і номерів рядків.

56.Чому secrets не можна записувати в logs?

Це призведе до витоку конфіденційних даних (паролів, токенів, ключів доступу) у файли логів.

57.Що таке streaming I/O?
Потокове читання або запис даних порціями чи по черзі, без завантаження всього об'єкта в оперативну пам'ять.

58.Яка перевага streaming для великих files?
Дозволяє обробляти файли, які значно перевищують обсяг доступної оперативної пам'яті (RAM).

59.Що таке eager file loading?
Завантаження всього файлу цілком в оперативну пам'ять однією операцією (наприклад, file.read() або readlines()).

60.Чому list(reader) може бути небажаним?
Завантажує весь вміст ітератора в пам'ять списком, нівелюючи переваги потокової обробки.

61.Що таке strict validation mode?
Режим роботи, за якого програма негайно зупиняється (кидає виняток) при знаходженні першого ж невалідного запису.

62.Що таке tolerant mode?
Режим обробки, за якого невалідні записи пропускаються з логуванням попередження, а програма продовжує обробку решти даних.

63.Коли доцільно використовувати strict mode?
Коли критично важлива стовідсоткова цілісність даних і жодна помилка вхідного файлу не допускається.

64.Коли tolerant mode?
Коли потрібно обробити великий масив «брудних» даних, зібравши коректні записи та зафіксувавши статистику по збоях.

65.Що таке atomic write?
Метод безпечного збереження файлу через тимчасовий файл із подальшою неподільною заміною основного, що гарантує відсутність «битих» файлів при збоях.

66.Для чого використовується temporary file?
Як безпечний буфер для запису даних перед фінальним перенесенням на місце цільового файлу.

67.Що може статися при звичайному output write та аварійному завершенні?
Файл може записатися наполовину, що призведе до його пошкодження (corruption) та втрати даних.

68.Навіщо видаляти temporary file після error?
Щоб не захаращувати файлову систему сміттям і не залишати неповні чи зайві файли в директорії.

69.Що таке JSON Lines?
Формат, де кожен рядок файлу є самостійним, валідним JSON-об'єктом, розділеним символом переведення рядка.

70.Чому JSON Lines зручний для streaming?
Дозволяє читати та обробляти великі масиви структурованих даних рядок за рядком без парсингу всього файлу як єдиного масиву.

Висновок: Під час виконання лабораторної роботи №5 було розроблено модуль імпорту, експорту та конфігурації даних для системи банківських рахунків.
У ході роботи було застосовано обробку винятків, pathlib, контекст-менеджери та генератори для ефективної роботи з файлами. Реалізовано завантаження конфігурації з YAML, потокове читання CSV, атомарний запис JSON і логування подій. Також забезпечено перевірку даних, обробку помилок і збереження цілісності файлів. Проєкт протестовано в PowerShell, а його код збережено в Git і завантажено на GitHub. У результаті отримано надійний модуль, який можна використовувати для подальшого розвитку системи банківських рахунків.